

10/10
50/10

Result
50/10

Generations of programming languages :

1G

Machine language or a binary language

2G

Assembly-level language: opcode operation codes to be fed, more of a hardware language

3G

High-level programming language, English-like language, translated and gave us WORA(Write Once Run Anywhere).

4G

SQL(Structured Query Language, a specialized language for databases, capable of giving CRUD)

5G

Natural language processing (NLP), prompt engineering, and AI.

C++:3g

C++ is a successor of C, which was procedure-oriented. This had optional object-oriented programming. That means in C++ you can write code in classes and objects but can also write normal code. Currently the latest version of C++ is 2021 and it has been upgraded from 1983 till now.

Working:

Job of compiler and interpreter :

A compiler and interpreter work as a translator from a high-level English-based language to a low-level machine-understandable language.

It works in multiple passes or in multiple stages.

It tokenizes the data, converts it to needed object code, and later converts it to the needed machine's native code(Machine's own).

It's a compilation-based language where code is first compiled and object code is created. Later on object code is fed and compiled back for running or an EXE file.

Language:

Alphabets were the first thing.

A-Z

Programming:

character set: ASCII(8bit):Eng

A-Z,a-z,0-9,!@#\$%&.....

Words .

Sentences, which are collections of words

Paragraphs, essay, etc.

Token: Words made from C++ character set

Datatypes:

Will describe the property of data one can store .

only numbers(Natural numbers +ve and -ve)

byte 1B

short 2B ✓

int 4B

long int 8B

Real numbers with a decimal point

float 4B ✓

double 8B

long double 16B

Any character from the C++ character set (alphabets, symbols, or even digits) can fall in this.

char 'a' 1B ✓

string "amar" n-Bytes ✓

boolean

0(false)/1(true) ✓

Signed

$$-2^n/2 \text{ to } +2^n/2 - 1$$

Byte

$$-128 \text{ to } +127 (0.127)$$

In C++ input and output are taken by `cin` and `cout` statements.

cin>>var

cout<<var

C >> Var



```
cin>>var1>>var2>>var3;  
cout<<"message";  
cout<<var;//will print value stored in var  
cout<<"message"<<var<<"message";
```

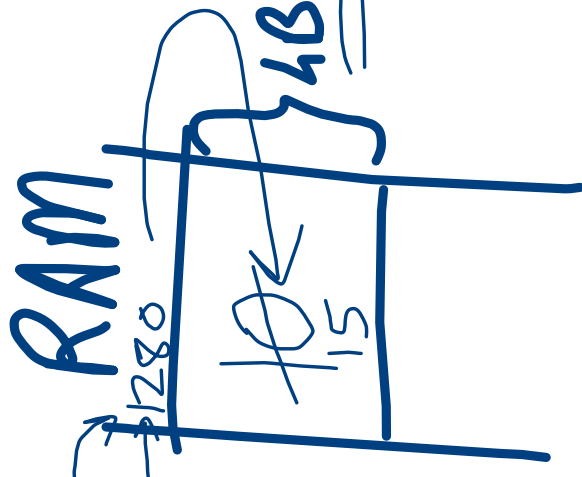
Variable identifier or naming conventions

- 1. Cannot use keyword.
int float;**
- 2. The first character cannot be a digit. Second onwards can be digit.
1a(wrong) a1(correct)**
- 3. Capital and lowercase alphabets are different. Case sensitivity: a!=A
int a=10;**

```
int a=10;
```

a=uts

a



Generally a memory is allocated to a variable and then the value is assigned to the variable, which will be stored in a memory allocator.

$$\% \Rightarrow$$

It returns the remainder of division.

$$5 \% 3 = 2 \quad \begin{array}{r} 3 \overline{) 5} \\ \underline{-3} \\ 2 \end{array}$$

$$17 \% 2 = 1 \quad \begin{array}{r} 2 \overline{) 17} \\ \underline{-16} \\ 1 \end{array}$$

$$50 \% 118 = 50 \quad \begin{array}{r} 118 \overline{) 50} \\ \underline{0} \\ 50 \end{array}$$

$$10 \% 10 = 0 \quad \begin{array}{r} 10 \overline{) 10} \\ \underline{-10} \\ 0 \end{array}$$

Operators:

Arithmetic

+

-

*

/

%

High: * / % 1st
Low: + - 2nd

$$\frac{5+2-3+8}{7-3+8} \Rightarrow \frac{12}{12} = 1$$

$$5+2*3-6 = 5+\underline{6}-6 = 11-6 = 5$$

$$((5+2)*(3-6)) \\ (7*-3) \\ -21$$

$$10+5*2 \quad 20$$

$$20-8/2+3=19$$

$$15+6*2-8/4=25$$

$$(10+5)*2-8/4=19/28$$

$$15*2-8/4$$

$$30-8/4$$

$$30-2$$

$$28$$

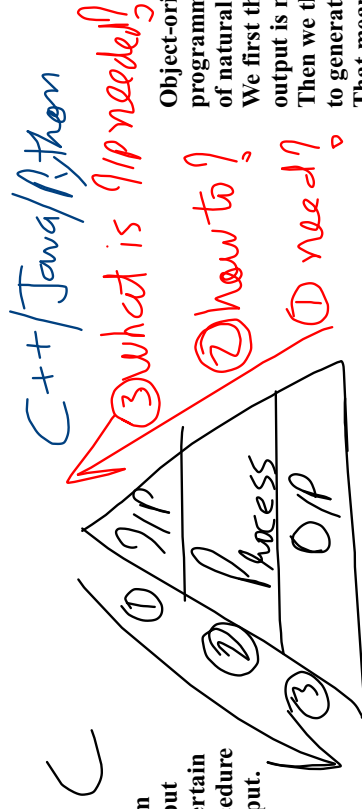
$$25-5*2+12/3\%2=15/17$$

$$⑥①⑤⑦③$$

$$25-10+12/3\%2=25-10+4\%2 \rightarrow 25-10+0=15//$$

$$\text{use int } 5/2 \rightarrow 2.5 \rightarrow 2$$

A procedure-oriented system talks about input subjected to certain steps or a procedure producing output.



Object-oriented programming is more natural to humans than anything else because humans are driven by the output needed or the objective to achieve.

60- 40
design code+testing

Object-oriented programming is a process of natural thinking. We first think about what output is needed. Then we think about how to generate this output. That means process. And then in order to perform that process, we need to think about what input would be needed.

Write a program to print the area of a circle.
1:area-->3.14*r*r--->read r

3.14*r*r);

Convert the given amount in INR to USD.

₹100 is US \$1.

:

o/p:in USD

process:inr/100

i/p:inr

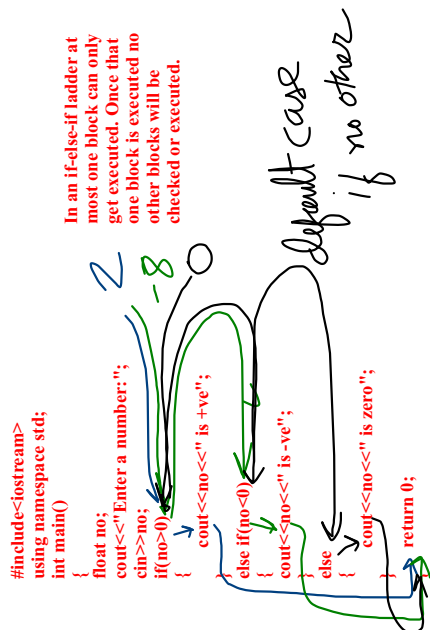
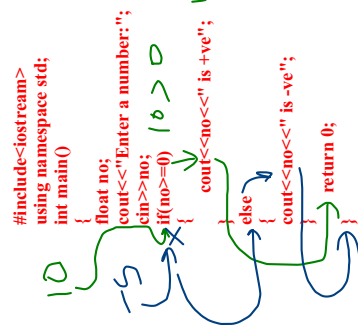
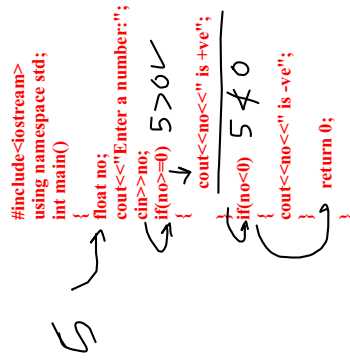
conditional :Branch

In a conditional statement we take a condition, either complex or normal, and then take a decision on right/wrong, true/false.

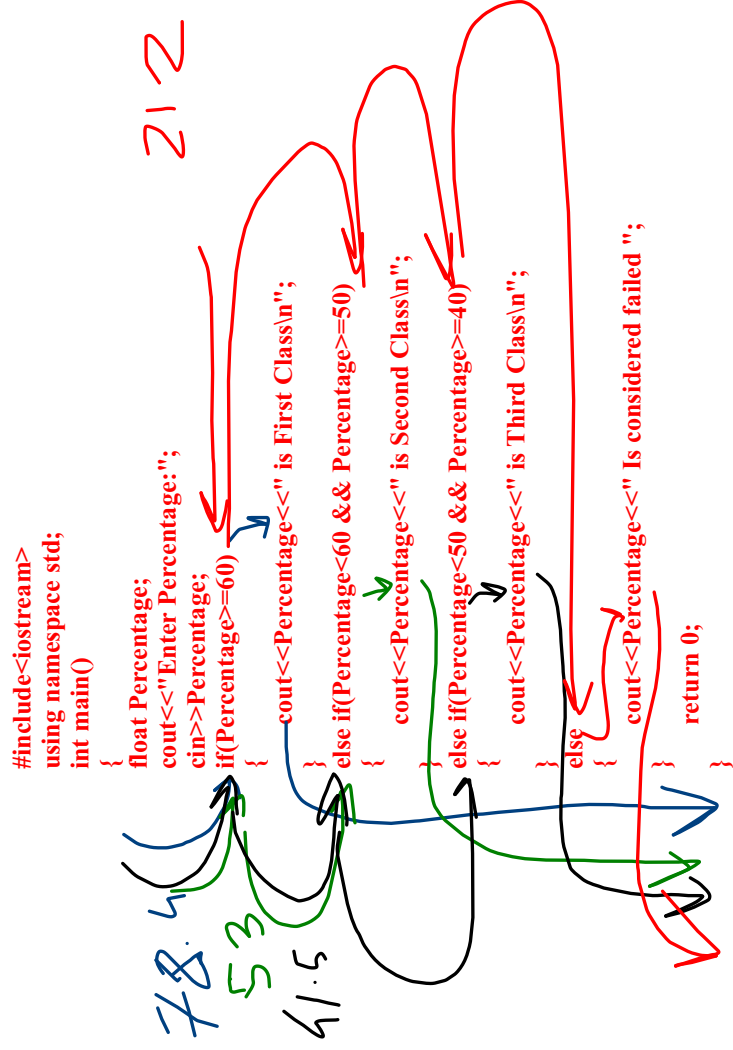
condition ?---->True :Action of step A.

|----->False:Action of step B.

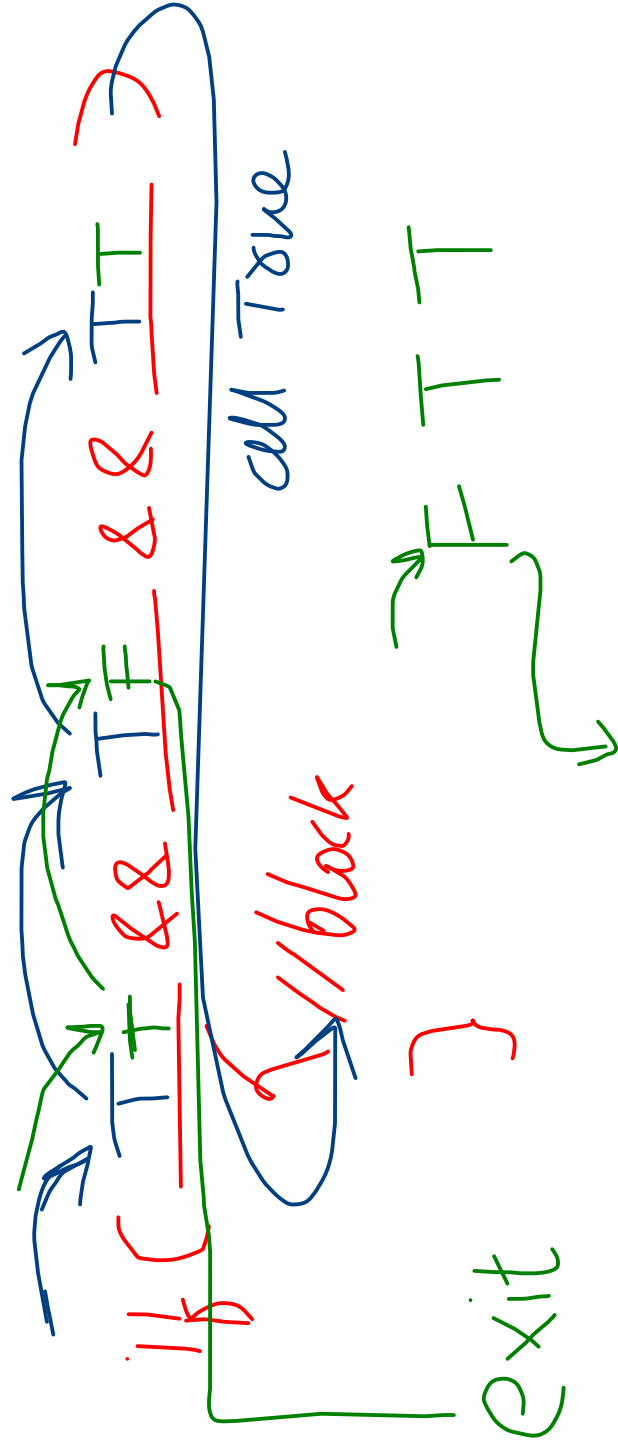
—

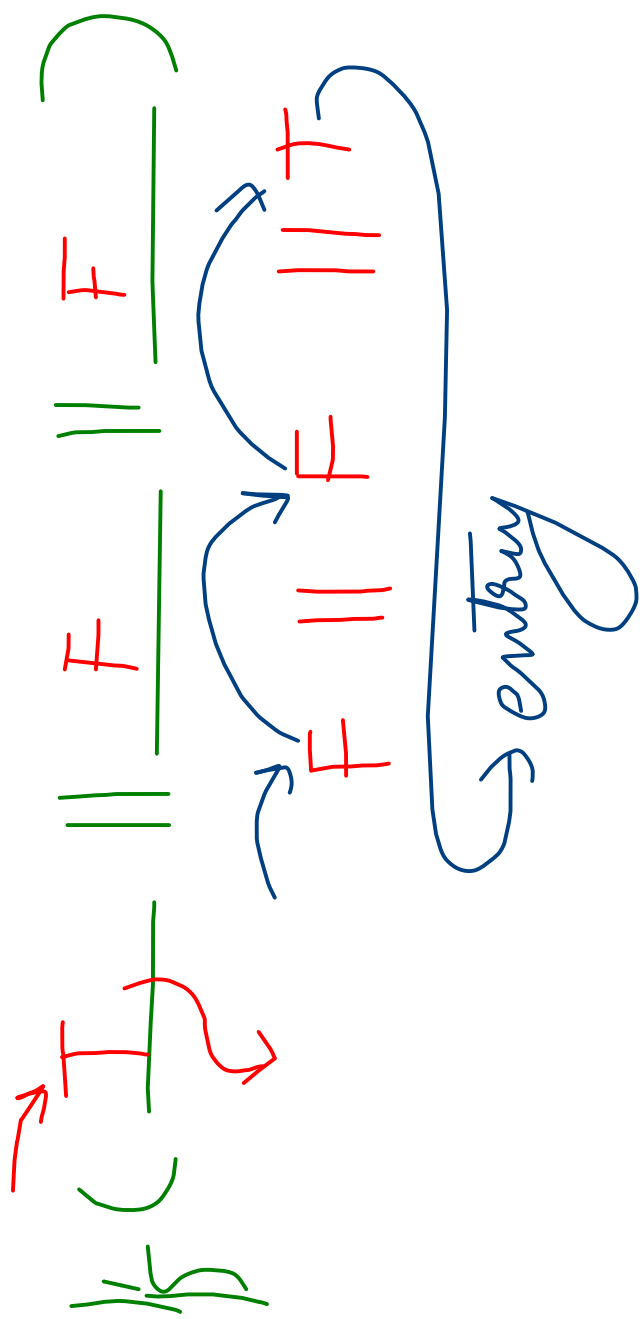


In an if-else-if ladder at most one block can only get executed. Once that one block is executed no other blocks will be checked or executed.



We have relational and logical. Relational gives true and false but logical can combine multiple complex relations into one.





&&:

In the and(&&) we will only enter the case if all conditions are evaluated to be true. If even one is negative (that means false), we will stop further checking and exit straight away.

c

0/990-0100-0
T/20-0100-0